

KNN_scratch

August 25, 2026

1 Ý tưởng của thuật toán KNN - lazy learning

- Tính toán khoảng cách từ samples test đến các samples train sau đó chọn ra k samples gần với samples test nhất
- Với bài toán classification - major voting
- Với bài toán regression - mean of labels of k samples closest
- Trong lúc train thuật toán không tính toán hay optimize tham số nào mà nhiệm vụ chính chỉ là lưu data train
- Mọi tính toán được thực hiện lúc inference

Lưu ý: - Do cần tính toán khoảng cách giữa các samples dựa trên các features của chúng nên cần normalization về cùng một khoảng giá trị để tránh khoảng cách bị chi phối hoàn toàn bởi feature có khoảng giá trị lớn

2 Phân tích toán học & Tối ưu tính toán

- Tập dữ liệu train: $X \in \mathbb{R}^{N \times d}$
- Mẫu test: $Z \in \mathbb{R}^{M \times d}$

Công thức khoảng cách Euclidean bình phương từ z đến mẫu x_i :

$$\|z - x_i\|_2^2 = (z - x_i)^T(z - x_i) = \|z\|_2^2 + \|x_i\|_2^2 - 2x_i^T z$$

Tối ưu hóa khi tìm Top-K Nearest Neighbors ($\arg \min_{x_i}$):

1. Thành phần $\|z\|_2^2$: Cố định với mọi x_i trong cùng 1 lượt test $\rightarrow$ **Bỏ qua**.
2. Thành phần $\|x_i\|_2^2$: Thuộc tập train $\rightarrow$ **Tính trước (pre-compute) và lưu cache**.
3. Thành phần $-2x_i^T z$: Tính toán trực tiếp bằng phép nhân ma trận Xz .

Bài toán thu gọn thành:

$$\arg \min_{x_i} \|z - x_i\|_2^2 \iff \arg \min_{x_i} (\|x_i\|_2^2 - 2x_i^T z)$$

Trường hợp đặc biệt (Đã L2-Normalize dữ liệu): Khi các vector có $\|x_i\|_2 = 1$ và $\|z\|_2 = 1$: - $\|x_i\|_2^2 = 1$ (Hằng số $\rightarrow$ Bỏ qua). - Khi đó:

$$\arg \min_{x_i} \|z - x_i\|_2^2 \iff \arg \min_{x_i} (-2x_i^T z) \iff \arg \max_{x_i} (x_i^T z)$$

Nhận xét: Minimize khoảng cách Euclidean lúc này tương đương với Maximize Cosine Similarity ($x_i^T z$).

```
[1]: import numpy as np

class KNearestNeighbor():
    def __init__(self, k = 5, num_classes = 3, method = 1):
        self.k = k
        self.num_classes = num_classes
        self.method = method
        self.eps = 1e-8 # avoid result == 0

    def train(self, X_train, y_train):
        self.X_train = X_train
        self.y_train = y_train.astype(int) # Đảm bảo label int cho np.bincount

        if self.num_classes == 3:
            self.num_classes = len(np.unique(y_train))

    def predict(self, X_test):
        if self.method == 1:
            distances = self._compute_distance_naive(X_test)
        elif self.method == 2:
            distances = self._compute_distance_medium(X_test)
        elif self.method == 3:
            distances = self._compute_distance_fast(X_test)
        return self._predict_label(distances)

    def _compute_distance_naive(self, X_test): # Naive two loop, inefficient way
        num_test = X_test.shape[0]
        num_train = self.X_train.shape[0]

        distances = np.zeros((num_test, num_train))

        for i in range(num_test):
            for j in range(num_train):
                distances[i, j] = np.sqrt(self.eps + np.sum((X_test[i, :] - self.
↪X_train[j, :])**2))
        return distances

    def _compute_distance_medium(self, X_test):
        num_test = X_test.shape[0]
        num_train = self.X_train.shape[0]

        distances = np.zeros((num_test, num_train))

        for i in range(num_test):
```

```

        diff = np.sum((self.X_train - X_test[i])**2, axis = 1) #  $N \times d - 1 \times d$ 
        distances[i] = np.sqrt(diff+self.eps)

    return distances

def _compute_distance_fast(self, X_test):
    Z_square = np.sum(X_test**2, axis = 1, keepdims=True) #  $M \times d \rightarrow M \times 1$ 
    X_square = np.sum(self.X_train**2, axis = 1, keepdims=True).T #  $N \times d \rightarrow N \times 1$ 
    ↪  $1 \times N$ 
    ZT = np.dot(X_test, self.X_train.T) #  $M \times d, d \times N \rightarrow M \times N$ 

    dists = Z_square + X_square + self.eps - 2*ZT

    return np.sqrt(np.maximum(dists, 0))

def _predict_label(self, distances):
    num_test = distances.shape[0]
    y_pred = np.zeros(num_test)

    for i in range(num_test):
        k_nearest_idx = np.argsort(distances[i])[:self.k]
        k_nearest_label = self.y_train[k_nearest_idx]
        y_pred[i] = np.argmax(np.bincount(k_nearest_label, minlength = self.
    ↪ num_classes))
    return y_pred

```

```

[ ]: X_train = np.array([[1, 1], [3, 1], [1, 4], [2, 4], [3, 3], [5, 1]],
    ↪ dtype=float)
    y_train = np.array([0, 0, 0, 1, 1, 1])

    X_test = np.array([[2, 2], [3, 4]], dtype=float)

    knn = KNearestNeighbor(k=1, num_classes = 10, method = 1)
    knn.train(X_train, y_train)
    print(knn.predict(X_test))

```

[0. 1.]

```

[3]: import numpy as np
    from sklearn.datasets import load_iris
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score
    from sklearn.neighbors import KNeighborsClassifier

```

3 Data Iris

```
[4]: # Load data Iris
iris = load_iris()
X, y = iris.data, iris.target

# Train test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state=18, stratify = y)

# Normolize
mean = np.mean(X_train, axis = 0)
std = np.mean(X_train, axis = 0)

X_train_scaled = (X_train - mean)/std
X_test_scaled = (X_test - mean)/std
```

```
[5]: # Initial model KNN
knn = KNearestNeighbor(k = 5,num_classes=3, method = 3)
knn.train(X_train_scaled, y_train)

# predict
y_pred = knn.predict(X_test_scaled)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print(f"Độ chính xác của model trên tập Iris test: {acc*100:.2f}")
```

Độ chính xác của model trên tập Iris test: 93.33

```
[6]: # Test 5 mẫu
print("Thực tế: ", y_test[:10])
print("Dự đoán: ", y_pred[:10].astype(int))
```

Thực tế: [0 1 0 0 2 1 1 0 2 1]

Dự đoán: [0 1 0 0 2 2 1 0 2 1]

```
[7]: import time
# Đo thời gian chạy cho từng method
methods = {
    1: "Naive (2 loops)",
    2: "Medium (1 loop)",
    3: "Fast (Vectorized - No loops)"
}

execution_times = {}
```

```

for method_id, name in methods.items():
    knn = KNearestNeighbor(k=5, method=method_id, num_classes=3)
    knn.train(X_train_scaled, y_train)

    start_time = time.time()
    knn.predict(X_test_scaled)
    end_time = time.time()

    elapsed_time = end_time - start_time
    execution_times[name] = elapsed_time
    print(f"[{name}]: {elapsed_time:.4f} giây")

```

[Naive (2 loops)]: 0.0143 giây
 [Medium (1 loop)]: 0.0030 giây
 [Fast (Vectorized - No loops)]: 0.0007 giây

```

[8]: # Dùng thư viện
knn_lib = KNeighborsClassifier(n_neighbors = 5)
knn_lib.fit(X_train_scaled, y_train)

# predict
start_time = time.time()
y_pred = knn_lib.predict(X_test_scaled)
end_time = time.time()
elapsed_time = end_time - start_time

print(f"Thời gian thực hiện: {elapsed_time}")

# Accuracy
acc = accuracy_score(y_test, y_pred)

print(f"Độ chính xác khi dùng thư viện: {acc*100:.2f}")

```

Thời gian thực hiện: 0.0019223690032958984
 Độ chính xác khi dùng thư viện: 93.33

```

[9]: # Test 5 mẫu
print("Thực tế: ", y_test[:10])
print("Dự đoán: ", y_pred[:10].astype(int))

```

Thực tế: [0 1 0 0 2 1 1 0 2 1]
 Dự đoán: [0 1 0 0 2 2 1 0 2 1]

4 Test Data

- Data Wine Dataset
- Breast_cancer Dataset
- Digit Dataset

```

[10]: import numpy as np
      from sklearn.datasets import load_wine, load_breast_cancer, load_digits
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import accuracy_score

[11]: wine = load_wine()

      X, y = wine.data, wine.target

[16]: breast_cancer = load_breast_cancer()

      X, y = breast_cancer.data, breast_cancer.target

[ ]: # digit = load_digits()

      # X, y = digit.data, digit.target

[12]: # split
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.8,
      ↪random_state = 28, stratify = y)

[13]: # Normalize
      mean = np.mean(X_train, axis = 0)
      std = np.std(X_train, axis = 0)

      X_train_scaled = (X_train - mean)/std
      X_test_scaled = (X_test - mean)/std

[14]: # Initial mode
      knn = KNearestNeighbor(k = 5, method = 3)
      # knn.train(X_train, y_train)
      knn.train(X_train_scaled, y_train)

[15]: # predict
      # y_pred = knn.predict(X_test)
      y_pred = knn.predict(X_test_scaled)

      acc = accuracy_score(y_test, y_pred)

      print(f"Độ chính xác: {acc*100:.2f}")

```

Độ chính xác: 96.50